

HYPERLEDGER BESU

Virtual Node Network Setup Guide

4-Node IBFT 2.0 Private Blockchain

Docker Compose · Besu v24.12.0 · Chain ID 1337

Personal Reference Document — May 2026

Overview & Architecture

This guide documents the exact steps to spin up a 4-node Hyperledger Besu private blockchain on a single machine using Docker Compose. The network uses IBFT 2.0 (Istanbul Byzantine Fault Tolerant) consensus, which requires 2/3+ validators to agree before producing a block.

Network Summary

Parameter	Value
Besu Version	24.12.0
Consensus	IBFT 2.0 (Istanbul BFT)
Chain ID	1337
Block Period	2 seconds
Number of Nodes	4 (all validators)
Fault Tolerance	1 node can fail (3/4 still reach 2/3+ quorum)
Gas Price	0 (free transactions)
Container Runtime	Docker Compose

Port Allocation

Node	Container Name	RPC HTTP Port	P2P Port	Role
Node 1	besu-node1	8545	30303	Boot node + Validator
Node 2	besu-node2	8546	30304	Validator
Node 3	besu-node3	8547	30305	Validator
Node 4	besu-node4	8548	30306	Validator

Directory Structure

After setup, your project folder looks like this:

```
~/besu-network/
├── genesis.json           ← Chain configuration (generated)
├── docker-compose.yml    ← Docker services definition
├── node1/
│   └── data/
│       ├── key           ← Node 1 private key
│       └── key.pub       ← Node 1 public key
├── node2/data/{key, key.pub}
├── node3/data/{key, key.pub}
└── node4/data/{key, key.pub}
```

Prerequisites

System Requirements

Requirement	Minimum	Your System
RAM	4 GB	7.6 GB ✓
Free Disk	10 GB	947 GB ✓
Java	Java 21	Java 21 ✓
Docker	20.x+	Installed ✓
Docker Compose	2.x+	Installed ✓

Verify Prerequisites

Run these checks before starting:

```
# Check Java version (must be 21)
java -version

# Check Docker
docker --version
docker compose version

# Check available RAM (need at least 4GB free)
free -h

# Check disk space
df -h ~
```

Step 1 — Install Besu

Download and install Besu v24.12.0 to /opt/besu:

```
cd ~
wget https://github.com/hyperledger/besu/releases/download/24.12.0/besu-24.12.0.tar.gz
tar xzf besu-24.12.0.tar.gz
sudo mv besu-24.12.0 /opt/besu
echo 'export PATH=$PATH:/opt/besu/bin' >> ~/.bashrc
source ~/.bashrc
```

Verify Installation

```
besu --version
# Expected output: besu/v24.12.0/linux-x86_64/...
```

✔ *If besu command is not found after install, run: source ~/.bashrc or open a new terminal.*

Step 2 — Create Project Structure

```
mkdir -p ~/besu-network/{node1,node2,node3,node4}/data
cd ~/besu-network
```

📝 *All subsequent commands assume you are inside ~/besu-network unless stated otherwise.*

Step 3 — Generate Validator Keys & Genesis Block

This single command generates cryptographic keys for all 4 nodes AND creates a genesis.json that lists them as validators. Run it from your home directory:

```
besu operator generate-blockchain-config \
  --config-file=/dev/stdin \
  --to=~/.besu-network/config-output \
  --private-key-file-name=key << 'EOF'
{
  "genesis": {
    "config": {
      "chainId": 1337,
      "berlinBlock": 0,
      "ibft2": {
        "blockperiodseconds": 2,
        "epochlength": 30000,
        "requesttimeoutseconds": 4
      }
    },
    "nonce": "0x0",
    "timestamp": "0x58ee40ba",
    "gasLimit": "0x47b760",
    "difficulty": "0x1",
    "coinbase": "0x0000000000000000000000000000000000000000000000000000000000000000",
    "alloc": {}
  },
  "blockchain": {
    "nodes": {
      "generate": true,
      "count": 4
    }
  }
}
EOF
```

Verify Output

```
ls ~/.besu-network/config-output/
# Should show: genesis.json keys/

ls ~/.besu-network/config-output/keys/
# Should show 4 folders (each is an Ethereum address like 0xABC...)
```

 Each folder name inside keys/ IS the Ethereum address of that validator node. Write these down!

Step 4 — Distribute Keys to Node Folders

Copy each node's unique key pair and the shared genesis file to the correct locations:

```
# Copy the genesis file
cp ~/.besu-network/config-output/genesis.json ~/.besu-network/genesis.json

# Get the 4 key folder names (these are the validator addresses)
KEY_DIRS=$(ls ~/.besu-network/config-output/keys/)

# Copy keys — node 1 gets KEY_DIRS[0], node 2 gets KEY_DIRS[1], etc.
for i in 1 2 3 4; do
  DIR=${KEY_DIRS[$((i-1))]}
  cp ~/.besu-network/config-output/keys/$DIR/key ~/.besu-network/node$i/data/key
  cp ~/.besu-network/config-output/keys/$DIR/key.pub ~/.besu-network/node$i/data/key.pub
done
```

```
echo "Node $i address: $DIR"
done
```

⚠ *Keep the private key files (key) secure. Anyone with a node's private key can impersonate that validator.*

Verify Keys Are in Place

```
ls ~/besu-network/node1/data/
ls ~/besu-network/node2/data/
ls ~/besu-network/node3/data/
ls ~/besu-network/node4/data/
# Each should show: key  key.pub
```

Step 5 — Write docker-compose.yml

Create the Docker Compose file that defines all 4 node services:

```
cat > ~/besu-network/docker-compose.yml << 'EOF'
version: '3.8'

services:
  node1:
    image: hyperledger/besu:24.12.0
    container_name: besu-node1
    volumes:
      - ./genesis.json:/config/genesis.json
      - ./node1/data:/data
    command: >
      --data-path=/data
      --genesis-file=/config/genesis.json
      --rpc-http-enabled
      --rpc-http-api=ETH,NET,IBFT,WEB3
      --rpc-http-host=0.0.0.0
      --rpc-http-port=8545
      --p2p-port=30303
      --network-id=1337
      --host-allowlist="*"
      --rpc-http-cors-origins="*"
      --min-gas-price=0
    ports:
      - "8545:8545"
      - "30303:30303"
    networks:
      - besu-net

  node2:
    image: hyperledger/besu:24.12.0
    container_name: besu-node2
    volumes:
      - ./genesis.json:/config/genesis.json
      - ./node2/data:/data
    command: >
      --data-path=/data
      --genesis-file=/config/genesis.json
      --rpc-http-enabled
      --rpc-http-api=ETH,NET,IBFT,WEB3
      --rpc-http-host=0.0.0.0
      --rpc-http-port=8546
      --p2p-port=30304
      --network-id=1337
      --host-allowlist="*"
      --rpc-http-cors-origins="*"
      --min-gas-price=0
      --bootnodes=enode://$(cat node1/data/key.pub)@besu-node1:30303
    ports:
      - "8546:8546"
      - "30304:30304"
    networks:
      - besu-net
    depends_on:
      - node1

  node3:
    image: hyperledger/besu:24.12.0
    container_name: besu-node3
    volumes:
      - ./genesis.json:/config/genesis.json
      - ./node3/data:/data
    command: >
```

```

--data-path=/data
--genesis-file=/config/genesis.json
--rpc-http-enabled
--rpc-http-api=ETH,NET,IBFT,WEB3
--rpc-http-host=0.0.0.0
--rpc-http-port=8547
--p2p-port=30305
--network-id=1337
--host-allowlist="*"
--rpc-http-cors-origins="*"
--min-gas-price=0
--bootnodes=enode://$(cat node1/data/key.pub)@besu-node1:30303
ports:
- "8547:8547"
- "30305:30305"
networks:
- besu-net
depends_on:
- node1

node4:
  image: hyperledger/besu:24.12.0
  container_name: besu-node4
  volumes:
    - ./genesis.json:/config/genesis.json
    - ./node4/data:/data
  command: >
    --data-path=/data
    --genesis-file=/config/genesis.json
    --rpc-http-enabled
    --rpc-http-api=ETH,NET,IBFT,WEB3
    --rpc-http-host=0.0.0.0
    --rpc-http-port=8548
    --p2p-port=30306
    --network-id=1337
    --host-allowlist="*"
    --rpc-http-cors-origins="*"
    --min-gas-price=0
    --bootnodes=enode://$(cat node1/data/key.pub)@besu-node1:30303
  ports:
    - "8548:8548"
    - "30306:30306"
  networks:
    - besu-net
  depends_on:
    - node1

networks:
  besu-net:
    driver: bridge
EOF

```

 *Node 1 is the bootnode — nodes 2, 3, 4 connect to it first to discover the network. Node 1 does NOT need a `--bootnodes` flag.*

Step 6 — Start the Network

```
cd ~/besu-network
docker compose up -d
```

Wait about 15 seconds for all nodes to initialise and peer with each other.

Verify All Containers Are Running

```
docker compose ps
# All 4 containers should show status: running
```

Check Blocks Are Being Produced

Run this twice, 10 seconds apart — the block number should increase each time (blocks every 2 seconds):

```
curl -s -X POST http://localhost:8545 \
-H "Content-Type: application/json" \
-d '{"jsonrpc":"2.0","method":"eth_blockNumber","params":[],"id":1}'

# Expected: {"jsonrpc":"2.0","id":1,"result":"0x..."} (hex block number)
```

Check Peer Count

Node 1 should see 3 peers (nodes 2, 3, 4):

```
curl -s -X POST http://localhost:8545 \
-H "Content-Type: application/json" \
-d '{"jsonrpc":"2.0","method":"net_peerCount","params":[],"id":1}'

# Expected result: "0x3" (3 peers in hex)
```

✅ *If peer count is 0x0, wait 30 more seconds. Nodes need time to complete peer discovery via the bootnode.*

Check IBFT Validators

```
curl -s -X POST http://localhost:8545 \
-H "Content-Type: application/json" \
-d
'{"jsonrpc":"2.0","method":"ibft_getValidatorsByBlockNumber","params":["latest"],"id":1}'

# Should return an array with 4 Ethereum addresses
```

Day-to-Day Management Commands

Lifecycle

Action	Command
Start network	<code>cd ~/besu-network && docker compose up -d</code>
Stop network (keep data)	<code>cd ~/besu-network && docker compose down</code>
Stop + wipe all data	<code>cd ~/besu-network && docker compose down -v</code>
Restart a single node	<code>docker restart besu-node2</code>
View logs (all nodes)	<code>docker compose logs -f</code>
View logs (one node)	<code>docker compose logs -f node1</code>
Check running containers	<code>docker compose ps</code>

Useful RPC Calls

Purpose	Command
Current block number	<code>curl -s -X POST http://localhost:8545 -H "Content-Type: application/json" -d '{"jsonrpc":"2.0","method":"eth_blockNumber","params":[],"id":1}'</code>
Node 1 peer count	<code>curl -s -X POST http://localhost:8545 -H "Content-Type: application/json" -d '{"jsonrpc":"2.0","method":"net_peerCount","params":[],"id":1}'</code>
List validators	<code>curl -s -X POST http://localhost:8545 -H "Content-Type: application/json" -d '{"jsonrpc":"2.0","method":"ibft_getValidatorsByBlockNumber","params":["latest"],"id":1}'</code>
Get chain ID	<code>curl -s -X POST http://localhost:8545 -H "Content-Type: application/json" -d '{"jsonrpc":"2.0","method":"eth_chainId","params":[],"id":1}'</code>

Troubleshooting

Problem: Containers exit immediately on startup

```
# Check logs for the failing container
docker logs besu-node1
```

Common causes:

- Wrong genesis file path — make sure genesis.json is at ~/besu-network/genesis.json
- Key files missing — check all 4 node data directories have key and key.pub
- Port already in use — another process may be on 8545. Use: `lsof -i :8545`

Problem: Blocks not increasing

```
# Check if nodes can see each other
docker exec besu-node1 curl -s -X POST http://localhost:8545 \
  -H "Content-Type: application/json" \
  -d '{"jsonrpc": "2.0", "method": "net_peerCount", "params": [], "id": 1}'
```

IBFT 2.0 needs at least 3 of 4 validators to produce blocks. If peer count is 0x0 or 0x1, consensus cannot be reached.

Problem: Need to start completely fresh

⚠ This deletes ALL blockchain data. You will need to regenerate keys and genesis from Step 3.

```
cd ~/besu-network
docker compose down
rm -rf node1/data/* node2/data/* node3/data/* node4/data/*
rm -f genesis.json docker-compose.yml
rm -rf config-output/
# Now go back to Step 3 and repeat from there
```

Problem: Docker image not found

```
# Pull the image manually
docker pull hyperledger/besu:24.12.0
```

Recovery — Starting From Scratch

If you lose your code but still have this document, follow these steps exactly in order. The key difference from a fresh setup is that you will generate NEW keys (new validator identities).

Quick Recovery Checklist

- Install Besu v24.12.0 (Step 1)
- Create project structure (Step 2)
- Generate fresh keys + genesis (Step 3) — new keys = new validator addresses
- Distribute keys to node folders (Step 4)
- Write docker-compose.yml (Step 5) — copy exactly
- Start with docker compose up -d (Step 6)
- Verify blocks are increasing and peer count is 0x3



Your new network will have a different genesis block and different validator addresses, but the behaviour is identical. Smart contracts need to be redeployed on the new chain.

Key Concepts Quick Reference

Term	What It Means
IBFT 2.0	Istanbul Byzantine Fault Tolerant — the consensus algorithm. Validators take turns proposing blocks; 2/3+ must sign for a block to be accepted.
Validator	A node whose key is listed in genesis.json as authorised to vote on and produce blocks.
Bootnode	The first node others connect to for peer discovery. Node 1 serves this role here.
Enode	A network identifier for a node: enode://<public-key>@<host>:<port>
Chain ID	1337 — a local/test chain ID. Prevents replay attacks across chains.
Genesis block	Block 0 — defines the chain rules, validator set, and initial state.
RPC HTTP	The JSON-RPC API used to query and send transactions to a node.
P2P port	The port nodes use to communicate with each other (not exposed to end users).

Appendix — genesis.json Structure Explained

The genesis.json is generated automatically in Step 3, but here is what each field means:

```
{
  "config": {
    "chainId": 1337,           // Network identifier (1337 = local dev)
    "berlinBlock": 0,         // Enable Berlin EVM rules from block 0
    "ibft2": {
      "blockperiodseconds": 2, // New block every 2 seconds
      "epochlength": 30000,    // Validator vote tallying interval
      "requesttimeoutseconds": 4 // How long to wait before timeout
    }
  },
  "extraData": "0x...",      // Encodes the initial validator addresses
  "gasLimit": "0x47b760",    // Max gas per block (~4.7M)
  "difficulty": "0x1",       // Always 1 for IBFT (PoA)
  "alloc": {}                // Pre-funded accounts (empty here)
}
```

 The `extraData` field is the most critical — it lists all 4 validator addresses. It is automatically generated by the besu operator `generate-blockchain-config` command.